Лабораторные работы №1–5 с использованием нейросетей

Анализ требований, Git, архитектура, интерфейс и база данных

Дисциплина: «Технологии разработки программного обеспечения»
Форма обучения: заочная

Как работать с методичкой

- Все работы выполняются на одном сквозном проекте «Система учёта заявок».
- Работайте самостоятельно и сохраняйте изменения в Git после каждого заметного этапа.
- Текст отчёта пишите своими словами; скриншоты должны показывать именно ваш проект.
- Для программной части рекомендуется Python 3.12+, Git, GitHub или GitLab, draw.io/diagrams.net. Если преподаватель разрешил другой стек, логика заданий сохраняется.

Общий проект и правила

Система учёта заявок должна позволять регистрировать обращения, хранить сведения о заявителе, категории и статусе, назначать исполнителя и контролировать выполнение.

Элемент	Минимальное требование
Пользователи	Администратор, оператор, исполнитель, заявитель
Заявка	Номер, тема, описание, дата, категория, статус, исполнитель
Статусы	Новая, В работе, Выполнена, Закрыта
Категории	Не менее 4 категорий
Репозиторий	Публичный или доступный преподавателю GitHub/GitLab

Правила использования нейросетей в курсе

Нейросети в этом курсе разрешены и рассматриваются как инструмент разработчика: для анализа, генерации черновиков, поиска ошибок, объяснения кода и критики решений. Оцениваться будет не сам факт генерации, а умение правильно поставить задачу, выбрать модель, проверить результат и осознанно доработать его.

Для каждой лабораторной студент обязан сохранить минимум один содержательный промпт и указать точное название использованной модели. Скриншот интерфейса нейросети или экспорт диалога прикладывается к отчёту, если это позволяет сервис.

Для каждого запроса необходимо указать: 1) какую задачу решали; 2) полный текст промпта; 3) модель; 4) почему выбрали именно эту модель и формулировку промпта; 5) что получила нейросеть; 6) что из результата использовано; 7) как результат проверен; 8) какие ошибки/неточности обнаружены и что исправил студент самостоятельно.

Запрещается без проверки выдавать ответ нейросети за собственную разработку. Студент отвечает за корректность требований, кода, SQL, тестов и документации. Не передавайте в нейросеть пароли, токены, персональные данные и другие секреты проекта.

Шаблон AI-журнала

Во всех лабораторных используйте один и тот же шаблон AI-журнала. Не вставляйте его повторно в каждый раздел: в отчёте достаточно заполнить запись для конкретной лабораторной. Минимум — одна содержательная запись на лабораторную; для нескольких значимых запросов добавляйте отдельные записи.

Поле	Что указать
1. Задача	Что вы хотели получить от нейросети и на каком этапе лабораторной использовали её.
2. Модель	Точное название модели и, если сервис показывает, версия/режим.
3. Промпт	Полный текст запроса без сокращений.
4. Почему такой промпт	Почему сформулировали запрос именно так; какие контекст, роль, формат, ограничения или критерии качества задали.
5. Почему такая модель	Почему выбрали эту модель для данной задачи: например, работа с кодом, диаграммами, длинным контекстом, анализом или критикой.
6. Результат ИИ	Что предложила нейросеть; при необходимости приложите скриншот или экспорт диалога.
7. Проверка	Что проверили самостоятельно: запуском кода, тестами, SQL, документацией, сравнением с требованиями и т. п.
8. Собственная доработка	Что изменили, исправили или отклонили и почему.
9. Вывод	Что вы узнали о работе с ИИ и качестве его результата при выполнении этой лабораторной.

Формат сдачи: таблица в отчёте или отдельный файл docs/ai_log.md. Скриншоты/экспорт диалога прикладываются по возможности; результат ИИ обязательно проверяется студентом самостоятельно.

Лабораторная работа №1. Анализ предметной области и требования

Цель работы

Научиться переводить исходную задачу в формализованные требования к программной системе.

Порядок выполнения

1. Создайте папку проекта с названием вида Фамилия_Имя_Группа_RequestSystem.

2. Создайте папки docs, src и tests. На этом этапе src и tests могут быть пустыми.

3. Изучите предметную область: кто создаёт заявку, кто её обрабатывает, кто меняет статус и кто контролирует результат.

4. Составьте краткое описание проблемы (1/2–1 страница): что происходит без системы и какую проблему решает программа.

5. Определите не менее 4 ролей пользователей и для каждой роли перечислите разрешённые действия.

6. Сформулируйте не менее 10 функциональных требований в формате «Система должна…».

7. Сформулируйте не менее 5 нефункциональных требований: удобство, надёжность, производительность, безопасность, сопровождаемость.

8. Выделите не менее 5 сущностей предметной области: пользователь, заявка, статус, категория, комментарий и т.п.

9. Сделайте диаграмму Use Case с ролями и основными вариантами использования.

10. Составьте список критериев приёмки: по каким признакам преподаватель сможет понять, что система соответствует требованиям.

11. Сохраните итоговое техническое задание в docs/TZ.md или docs/TZ.docx.

Минимальное содержание технического задания

Раздел	Что должно быть
1. Назначение	Для чего создаётся система и кто ей пользуется
2. Роли	Минимум 4 роли и их права
3. Функции	Минимум 10 функциональных требований
4. Нефункциональные требования	Минимум 5 требований
5. Сценарии	Минимум 5 Use Case
6. Приёмка	Критерии готовности

Задание по работе с нейросетями

1. Возьмите исходную задачу вашей системы и попросите нейросеть выступить аналитиком: выделить роли пользователей, функциональные и нефункциональные требования, сущности предметной области и возможные критерии приёмки.

2. Сформулируйте два разных промпта для одной и той же задачи: первый — свободный, второй — структурированный (с указанием роли, контекста, формата ответа и ограничений).

3. Сравните ответы и выберите один вариант. В отчёте объясните, почему выбранный промпт дал более полезный результат.

4. Выберите минимум 3 требования, предложенных ИИ, и проверьте их самостоятельно. Найдите хотя бы одну неточность или двусмысленность (если она действительно есть) и исправьте её.

5. Сдайте полный текст обоих промптов, название модели, объяснение выбора, краткое сравнение ответов и итоговый вариант требований. Нейросеть должна помогать вам анализировать, а не заменять анализ.

AI-журнал для лабораторной №1: заполните запись по шаблону «Шаблон AI-журнала» в начале методички. Сам шаблон в отчёт не копируйте.

Что отправить преподавателю

Техническое задание. • Диаграмма Use Case (PNG/PDF/документ). • Ссылка на Git-репозиторий. • Отчёт 5–7 страниц со скриншотами результата.

Самопроверка перед отправкой

☐ В требованиях нет противоречий.

☐ Каждая функция имеет понятного пользователя.

☐ Диаграмма соответствует тексту.

☐ Файлы лежат в репозитории.

Лабораторная работа №2. Создание и ведение Git-репозитория

Цель работы

Научиться организовывать разработку проекта с использованием системы контроля версий Git.

Порядок выполнения

1. Установите Git и проверьте его версию командой git --version.

2. Создайте репозиторий на GitHub или GitLab.

3. Создайте локальный репозиторий командой git init.

4. Настройте имя и e-mail разработчика в Git.

5. Создайте README.md с названием проекта, назначением и кратким описанием функций.

6. Создайте .gitignore и исключите из него виртуальное окружение, служебные файлы IDE, кэш Python и локальные базы/секреты.

7. Добавьте первоначальную структуру проекта и выполните первый commit.

8. Создайте минимум 5 осмысленных commit. Каждый commit должен отражать отдельный этап: требования, диаграммы, README, структура каталогов и т.д.

9. Создайте ветку develop и внесите в неё отдельное изменение.

10. Слейте develop в main/master. При возникновении конфликта разрешите его вручную и сохраните результат.

11. Отправьте изменения в удалённый репозиторий.

12. Проверьте, что преподаватель сможет открыть репозиторий и увидеть историю commit.

Рекомендуемые сообщения commit:

```
docs: add technical specification
docs: add use case diagram
chore: add project structure
docs: update README
chore: add gitignore
```

Задание по работе с нейросетями

1. Попросите нейросеть объяснить рекомендуемый Git-workflow для учебного проекта и предложить последовательность команд для создания репозитория, ветки develop, commit и merge.

2. Отдельно попросите нейросеть предложить содержательные сообщения для минимум 5 commit и шаблон .gitignore для выбранного стека.

3. Выполните команды самостоятельно и проверьте результат. Не копируйте команды вслепую: для каждой потенциально опасной команды (merge, reset, push --force и т.п.) объясните, что она делает.

4. Смоделируйте или разберите один типичный конфликт Git. Спросите у нейросети, как его диагностировать и исправить, затем опишите, что вы сделали сами.

5. В AI-журнале укажите модель, полные промпты, причину выбора и минимум один пример, где совет ИИ пришлось изменить после проверки.

AI-журнал для лабораторной №2: заполните запись по шаблону «Шаблон AI-журнала» в начале методички. Сам шаблон в отчёт не копируйте.

Что отправить преподавателю

Ссылка на репозиторий. • Скриншот истории commit. • Скриншот веток main/master и develop. • Отчёт с кратким описанием использованных команд Git.

Самопроверка перед отправкой

☐ Есть минимум 5 осмысленных commit.

☐ README открывается из репозитория.

☐ .gitignore создан.

☐ В репозитории нет паролей, токенов и других секретов.

☐ Изменения доступны преподавателю по ссылке.

Лабораторная работа №3. Проектирование архитектуры программного продукта

Цель работы

Научиться выделять слои, компоненты и границы ответственности программной системы.

Порядок выполнения

1. Возьмите список требований из лабораторной работы №1.

2. Выберите архитектурный вариант. Для учебного проекта рекомендуется слоистая архитектура: presentation/controller → service → repository/data access → database.

3. Разделите систему минимум на 4 логических слоя.

4. Для каждого слоя перечислите его ответственность и запретите ему выполнять чужую работу.

5. Определите основные модули: пользователи, заявки, статусы, категории, комментарии.

6. Создайте архитектурную диаграмму.

7. Создайте диаграмму компонентов.

8. Предложите структуру каталогов исходного кода.

9. Для 3–5 ключевых классов или модулей укажите назначение, входные данные и результат.

10. Объясните, почему выбранная архитектура подходит для проекта.

11. Сохраните диаграммы в docs/architecture и добавьте их в репозиторий.

Компонент	Ответственность	Не должен делать
Controller	Принимает запрос и передаёт его дальше	Не должен содержать SQL
Service	Реализует бизнес-правила	Не должен рисовать интерфейс
Repository	Читает/изменяет данные	Не должен принимать бизнес-решения
Database	Хранит данные	Не должна содержать UI-логику

Задание по работе с нейросетями

1. Перед проектированием архитектуры дайте нейросети ваши основные требования и попросите предложить минимум 2 архитектурных варианта.

2. Попросите модель сравнить варианты по критериям: простота, тестируемость, расширяемость, стоимость внедрения и соответствие учебному проекту.

3. Выберите архитектуру самостоятельно и внесите в отчёт аргументы «за» и «против». Ответ ИИ используйте как консультацию, а не как готовое решение.

4. После создания своей диаграммы попросите нейросеть выступить архитектурным ревьюером: найти нарушения разделения ответственности, слишком связанные компоненты и потенциальные проблемы.

5. Для каждого замечания укажите: принято / отклонено / исправлено. Обязательно приложите полный промпт и модель.

AI-журнал для лабораторной №3: заполните запись по шаблону «Шаблон AI-журнала» в начале методички. Сам шаблон в отчёт не копируйте.

Что отправить преподавателю

Архитектурная диаграмма. • Диаграмма компонентов. • Описание слоёв и структуры проекта. • Обновлённый Git-репозиторий. • Отчёт.

Лабораторная работа №4. Проектирование пользовательского интерфейса

Цель работы

Разработать понятный прототип интерфейса системы до реализации экранов в коде.

Порядок выполнения

1. Просмотрите требования к пользовательским сценариям из лабораторной №1.
2. Создайте новый файл прототипа в Figma, draw.io или другом инструменте.
https://www.figma.com/solutions/ai-design-generator/
3. Сделайте экран входа в систему.
4. Сделайте главный экран со списком заявок.
5. Сделайте экран создания заявки.
6. Сделайте карточку заявки с полной информацией.
7. Сделайте экран редактирования заявки.
8. Добавьте поиск и фильтрацию по статусу и категории.
9. Добавьте визуальное отображение статуса заявки.
10. Предусмотрите сообщения об ошибках и подтверждение опасных действий, например удаления.
11. Для каждого экрана укажите назначение и доступные действия пользователя.
12. Сохраните прототип и экспортируйте ключевые экраны в PNG/PDF либо отправьте ссылку на figma.

Задание по работе с нейросетями

1. Попросите нейросеть составить UX-чек-лист для системы учёта заявок: навигация, формы, ошибки, пустые состояния, фильтрация, доступность и понятность статусов.
2. Создайте свой прототип и затем попросите нейросеть критически оценить его по этому чек-листу. При возможности прикрепите скриншоты собственных экранов.
3. Выберите минимум 5 замечаний ИИ и для каждого решите, нужно ли его учитывать. Обоснуйте решение.
4. После доработки прототипа укажите, какие изменения были внесены именно по результатам работы с нейросетью.
5. В отчёт приложите промпт, название модели, причину выбора модели и краткую таблицу «замечание ИИ → моё решение → результат».

AI-журнал для лабораторной №4: заполните запись по шаблону «Шаблон AI-журнала» в начале методички. Сам шаблон в отчёт не копируйте.

Что отправить преподавателю

- Ссылка или файл прототипа.
- Не менее 5 экранов.
- Изображения экранов.
- Отчёт с пояснением решений интерфейса.

Самопроверка перед отправкой

☐ Из каждого экрана понятно, куда нажать дальше.
☐ Есть сценарий создания и просмотра заявки.
☐ Ошибки и пустые состояния предусмотрены.
☐ Прототип соответствует ролям и требованиям.

Лабораторная работа №5. Проектирование и создание базы данных

Цель работы

Научиться переходить от модели предметной области к реляционной базе данных и SQL-запросам.

Порядок выполнения

1. Определите сущности и связи на основе требований.
2. Создайте ER-диаграмму.

3. Спроектируйте минимум 5 таблиц: Users, Requests, Statuses, Categories, Comments. При необходимости добавьте Roles, RequestHistory, Attachments.
4. Для каждой таблицы определите первичный ключ и обязательные поля.
5. Определите внешние ключи и связи 1:M или M:M там, где это необходимо.
6. Проверьте структуру на дублирование данных и выполните базовую нормализацию.
7. Создайте SQL-скрипт schema.sql, который создаёт таблицы с нуля.
8. Создайте SQL-скрипт seed.sql с тестовыми данными.
9. Создайте SQLite-базу и выполните скрипты.
10. Сделайте не менее 5 INSERT, 5 SELECT, 2 UPDATE, 2 DELETE.
11. Сделайте не менее 3 SELECT с WHERE и не менее 2 запросов с JOIN.
12. Сохраните БД и все SQL-скрипты в репозитории.

Проверка	**Минимум**
Таблицы	5
INSERT	5
SELECT	5
UPDATE	2
DELETE	2
WHERE	3
JOIN	2

Задание по работе с нейросетями

1. Перед созданием БД дайте нейросети свои сущности и связи и попросите предложить ER-модель с таблицами, ключами и ограничениями.

2. Попросите модель подготовить черновой schema.sql и набор тестовых данных. Не используйте результат без проверки.

3. Проверьте вручную первичные и внешние ключи, типы данных, ограничения NOT NULL/UNIQUE, отсутствие очевидного дублирования и соответствие вашей ER-диаграмме.

4. Попросите нейросеть предложить сложные SQL-запросы для поиска и JOIN, затем выполните их в своей БД и проверьте результаты на реальных тестовых данных.

5. Минимум один SQL-фрагмент из ответа ИИ измените самостоятельно. В отчёте объясните, зачем изменили его.

6. Сдайте полный промпт, модель, причины выбора, исходный ответ, результаты проверки и финальные SQL-файлы.

AI-журнал для лабораторной №5: заполните запись по шаблону «Шаблон AI-журнала» в начале методички. Сам шаблон в отчёт не копируйте.

Что отправить преподавателю

- ER-диаграмма.
- schema.sql и seed.sql.
- Файл SQLite.
- Файл с SQL-запросами и результатами.
- Отчёт.

Самопроверка перед отправкой

☐ Внешние ключи настроены.

☐ SQL-скрипт можно выполнить повторно после очистки БД.

☐ Тестовые данные позволяют проверить функции приложения.
☐ Результаты запросов зафиксированы скриншотами.